



I²C – DAS WICHTIGSTE FÜR EINSTEIGER

Allgemeine Grundlagen plus Sensormeter/Sensormeter-PoE-spezifische Praxis für den RJ45-Modulbau

SDA/SCL · Pull-ups · Adressen

Ergänzt die modulspezifischen *-modul.md-Dokumente

github.com/peterhagelhof7-cmd/sensormeter-family

I²C (Inter-Integrated Circuit) ist ein einfacher serieller Datenbus, mit dem ein Mikrocontroller (hier: der ESP32 im Sensormeter bzw. Sensormeter PoE) viele Sensoren und andere Bausteine mit nur zwei Datenleitungen verbinden kann. Dieses Dokument erklärt zunächst die Grundlagen allgemein und geht dann konkret auf die Sensormeter-Family ein — inklusive einer praktischen Anleitung, wie man mit einem einfachen Multimeter prüft, ob ein vorliegendes Sensor-Modul bereits eigene Pull-up-Widerstände mitbringt.

1. Die zwei Leitungen

Ein I²C-Bus braucht immer:

- **SDA** (Serial Data) — Datenleitung
- **SCL** (Serial Clock) — Taktleitung
- zusätzlich: **GND** gemeinsam für alle Geräte
- eine Versorgungsspannung (meist 3,3 V oder 5 V)

Alle Geräte hängen parallel an denselben SDA- und SCL-Leitungen.

SENSORMETER-SPEZIFISCH

Am RJ45-Modulanschluss (Sensormeter WT32-ETH01 und Sensormeter PoE ESP32-S3-ETH, siehe [README.md](#) in diesem Verzeichnis) liegen SDA/SCL auf **Pin 4** bzw. **Pin 3**, 3V3 auf Pin 1, GND auf Pin 2 — bei beiden Geräten identisch (nur die geräteinternen GPIO-Nummern dahinter unterscheiden sich). Beide Geräte betreiben intern zusätzlich ein OLED-Display am selben I²C-Bus (0x3C) — der Bus ist also nie wirklich leer, selbst ohne gestecktes Modul.

2. Master und Slave

I²C arbeitet mit einem Master-Slave-Prinzip:

- **Master:** steuert den Bus (bei uns immer der ESP32)
- **Slave:** reagiert auf Befehle (Sensor, Display, Speicher usw.)

Beispiel: ESP32 = Master, BME280-Tempersensordaten = Slave.

SENSORMETER-SPEZIFISCH

Der ESP32 ist bei uns **immer** Master — es gibt keinen Betriebsmodus, in dem Sensormeter/Sensormeter PoE selbst als I²C-Slave auftreten. Auf dem Bus hängen bis zu drei Teilnehmer gleichzeitig: internes Display (0x3C, immer), optional ein externes SH1107-Anzeigemodul (0x3D) und optional genau ein Sensor-/Aktormodul.

3. Jedes Gerät braucht eine Adresse

Damit der Master weiß, welches Gerät gemeint ist, hat jedes I²C-Gerät eine Adresse, z. B.:

```
ESP32
|
+--- Sensor 1 (Adresse 0x76)
|
+--- Sensor 2 (Adresse 0x40)
|
+--- Display (Adresse 0x3C)
```

Wichtig: Zwei gleiche Geräte mit gleicher Adresse funktionieren nicht direkt zusammen. Lösung: Adresse ändern (falls das Modul einen Adress-Wahlpin hat), Multiplexer verwenden oder zweiten I²C-Bus nutzen.

In der Sensormeter-Family tatsächlich vorkommende Adressen

Adresse	Gerät/Chip	Firmware erkennt Name?
0x3C	Internes OLED-Display (SSD1306)	—, wird beim Scan bewusst übersprungen
0x3D	Externes Display-Modul (SH1107)	—, wird beim Scan bewusst übersprungen
0x38	AHT20/AHT21 (auch im AHT20+BMP280- und ENS160+AHT21-Kombimodul)	Ja — <code>KNOWN_CHIPS</code>
0x76 / 0x77	BMP280 bzw. BME280 (SDO-Pin wählt Adresse)	Nur als "BME280" — kein Chip-ID-Check, ein reines BMP280 wird fälschlich so benannt
0x23 / 0x5C	BH1750	Ja (Name), Lux-Wert wird aber noch nicht ausgelesen
0x52 / 0x53	ENS160	Nein — fehlt noch in <code>KNOWN_CHIPS</code> , erscheint als "unbekannt"
0x44 / 0x45	SHT30/31/35 (aktuell kein Familienmodul dafür entworfen)	Ja

REALER ADRESSKONFLIKT IN DIESER FAMILIE

Beim **AHT20+BMP280-Kombimodul** und beim **ENS160+AHT21-Kombimodul** sitzen zwei I²C-Chips auf derselben Platine. `SensorDetector.cpp` scannt aufsteigend ab 0x08 und bricht beim **ersten** Treffer ab (siehe `module-design/README.md`, Abschnitt "Kategorie 1"). Da 0x38 (AHT20/AHT21) numerisch kleiner ist als 0x52/0x53 (ENS160) und 0x76/0x77 (BMP280), gewinnt der AHT-Chip **immer** — der zweite Chip auf demselben Kombimodul ist für die aktuelle Firmware unsichtbar, auch wenn er physisch korrekt angeschlossen ist. Das ist keine Fehlfunktion, sondern die dokumentierte "genau ein Modul erwartet"-Designentscheidung.

4. Geschwindigkeit

Typische I²C-Geschwindigkeiten:

- 100 kHz → Standard, sehr zuverlässig
- 400 kHz → schneller, aber empfindlicher
- 10–50 kHz → sinnvoll bei langen Kabeln

Beim ESP32 kann die Geschwindigkeit per Software angepasst werden.

SENSORMETER-SPEZIFISCH — STAND 2026-07-15 (BITTE DIESE VERSION BEACHTEN)

Anders als teils angenommen läuft der Bus bei Sensormeter/Sensormeter PoE **nicht** fest auf 50 kHz. Die App-Firmware ruft nirgends `wire.setClock()` auf — der Takt wird ausschließlich von den Display-Bibliotheken pro Bildschirm-Update gesetzt: das interne SSD1306 erzwingt während des Schreibens 400 kHz und lässt danach 100 kHz stehen, das externe SH1107-Modul erzwingt sogar 1 MHz während des Schreibens. Effektiv läuft der Bus also die meiste Zeit bei **~100 kHz** (dem "Ruhetakt" nach dem letzten Display-Refresh) mit kurzen Ausschlägen auf 400 kHz/1 MHz. Eine feste 50-kHz-Begrenzung wurde am 2026-07-15 diskutiert, aber zurückgestellt (würde Änderungen an den Display-Bibliothek-Konstrukteuren brauchen und jeden Bildschirm-Refresh deutlich verlangsamen) — siehe `docs/entscheidungen.md` in `sensormeter/repo` und `sensormeter-poe/repo`, Eintrag "Periodischer I2C-Rescan (1x/Minute), ohne Taktänderung".

5. Kabellänge

I²C ist eigentlich für kurze Verbindungen gedacht:

- Leiterplatte: problemlos
- einige 10 cm: problemlos
- 1–2 m: oft machbar
- mehrere Meter: schwierig

Bei langen Leitungen helfen: langsamere Geschwindigkeit, bessere Verkabelung, Bus-Extender, oder alternativ RS-485/CAN für die größere Distanz.

SENSORMETER-SPEZIFISCH

Die RJ45-Steckmodule dieser Familie dürfen laut "Durchschleif-Regel" (`module-design/README.md`) maximal **einen** zusätzlichen Kategorie-1-Hop verketteten (Gerät → Modul, kein zweites I²C-Modul dahinter am selben Strang außer im Bus-Abgriff-Fall). Je länger die RJ45-Kabel zwischen den Hops, desto mehr Leitungskapazität — bei sichtbaren Fehlfunktionen (siehe Abschnitt 8) zuerst die Kabellänge prüfen, bevor an der Firmware gesucht wird.

6. Pull-up-Widerstände

I²C benötigt Pull-up-Widerstände an SDA und SCL. Sie sorgen dafür, dass die Leitungen im Ruhezustand wieder auf HIGH gehen (I²C-Geräte ziehen die Leitung aktiv nur nach LOW, nie aktiv nach HIGH). Typische Werte:

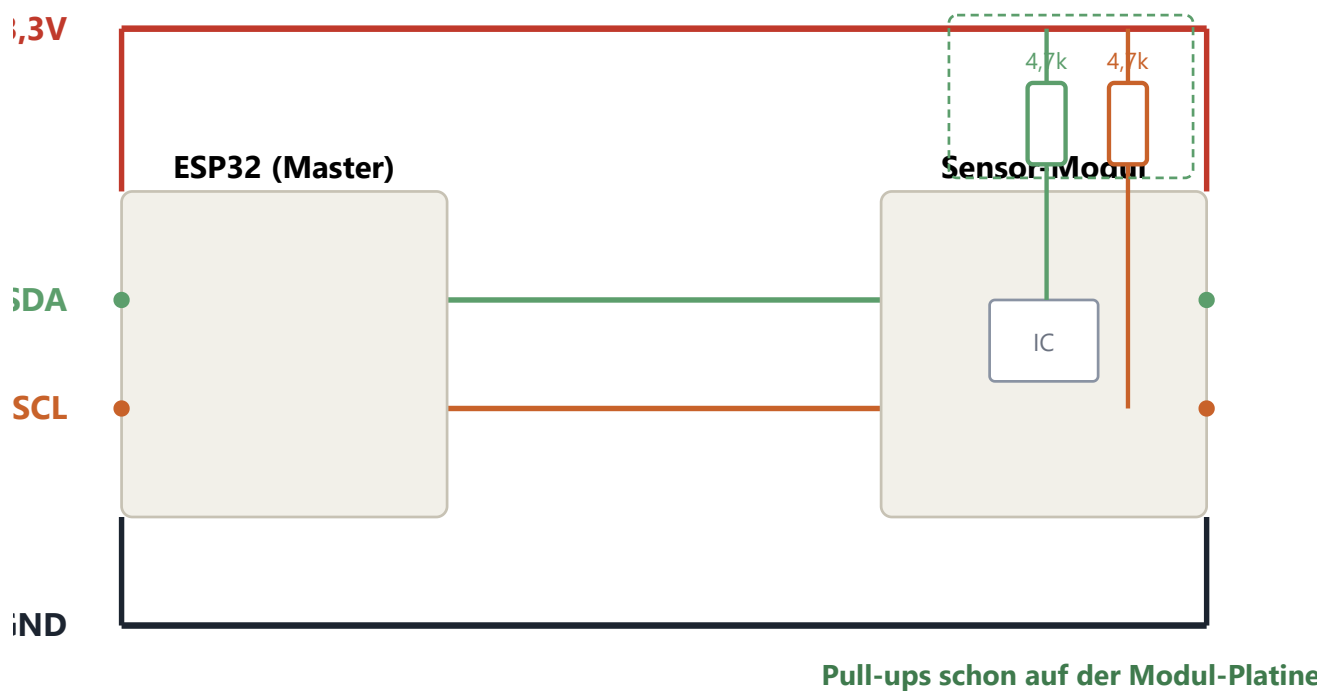
- 4,7 kΩ → häufig verwendet, auch der bei Sensormeter-Family-Modulen verwendete Wert
- 2,2 kΩ → für längere Leitungen oder viele Geräte
- zu viele Pull-ups parallel (mehrere Module gleichzeitig gesteckt) → sinkender effektiver Widerstand, kann Probleme verursachen

Viele fertige Sensor-Module haben bereits Pull-ups eingebaut — genau das lässt sich mit einem einfachen Multimeter überprüfen, siehe Anleitung unten.

SENSORMETER-SPEZIFISCH (FAMILIENREGEL)

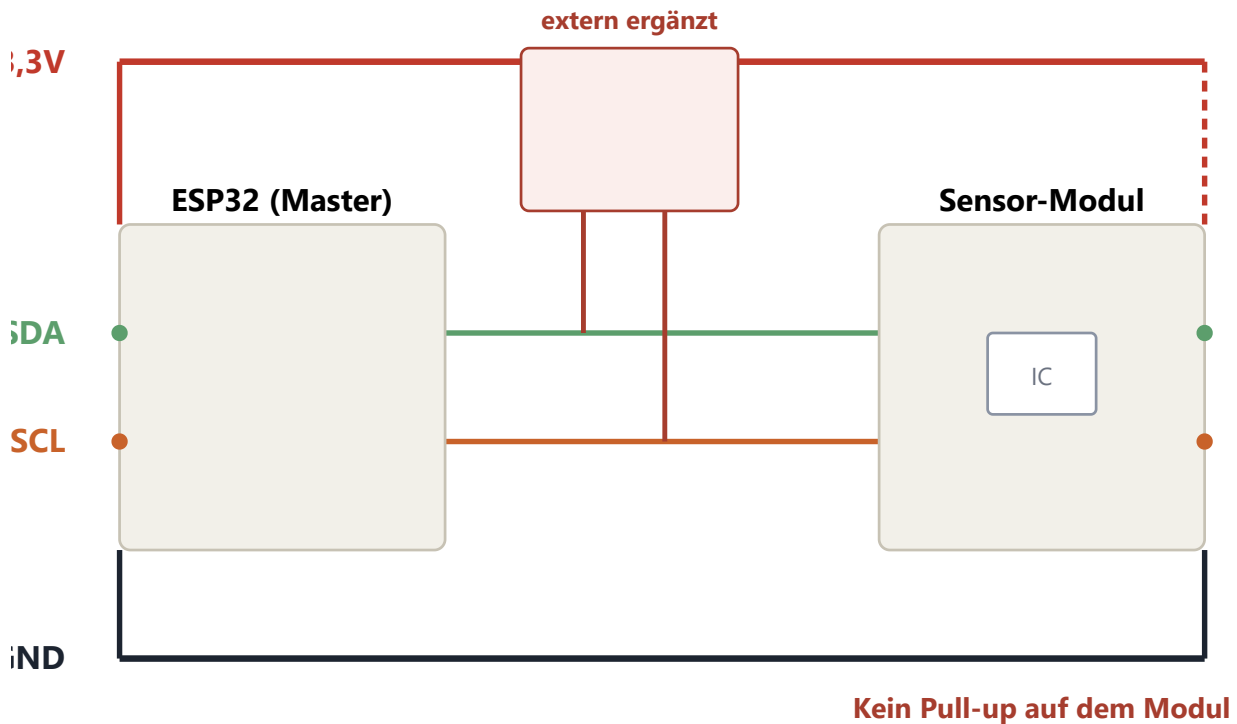
Nach der Konvention aus `module-design/README.md` sitzen Pull-ups grundsätzlich **auf dem Modul**, nicht auf dem Gerät — und **nur, falls der Bus dort noch keinen hat** (der gemeinsame Bus mit dem OLED-Display kann bereits einen mitbringen). Vor dem Bestücken eines neuen Moduls immer erst mit dem Multimeter prüfen (siehe unten) — ein doppelter Pull-up senkt den effektiven Widerstand unnötig.

Situation A — Modul bringt bereits einen Pull-up mit



Nichts weiter nötig — Modul direkt anschließen. Kein zusätzlicher Widerstand am Gerät oder in der Leitung.

Situation B — Modul hat KEINEN Pull-up



4,7 k Ω zwischen 3,3V und SDA sowie zwischen 3,3V und SCL ergänzen — z. B. auf der Steckmodul-Platine selbst (Familienkonvention) oder als kleine Zusatzplatine nahe der Buchse. Ohne diese Widerstände bleibt der Bus auf LOW hängen, keine Kommunikation möglich.

Anleitung: Mit dem Ohmmeter prüfen, ob ein Modul bereits einen Pull-up hat

VORAUSSETZUNG

Das Modul darf während der Messung an **keiner** Spannungsquelle hängen — weder im Gerät gesteckt noch anderweitig extern versorgt. Eine anliegende Spannung verfälscht die Messung oder lässt das Multimeter Unsinn anzeigen.

1. **Messgerät vorbereiten:** Multimeter auf Widerstandsmessung (Ω -Symbol) stellen — **nicht** auf den Durchgangsprüfer/„Diode“-Modus mit Piepton. Der Diodenmodus schlägt oft nur bei sehr niedrigen Werten ($\sim 30\text{--}50\ \Omega$) an und liefert bei k Ω -Widerständen ein irreführendes „kein Durchgang“ statt eines Messwerts.
2. **SDA gegen 3V3/VCC messen** (präziseste Methode): eine Messspitze an den 3V3/VCC-Pin des Moduls, die andere an SDA.
 - Anzeige ca. **1–10 k Ω** (typisch 2,2 / 4,7 / 10 k Ω) → Pull-up vorhanden.
 - Anzeige „OL“/unendlich bzw. mehrere M Ω → kein Pull-up auf dieser Leitung.
3. **Dieselbe Messung für SCL gegen 3V3/VCC wiederholen.** Beide Leitungen einzeln prüfen — manche Module haben nur auf einer Leitung einen Pull-up oder unterschiedliche Werte je Leitung.
4. **Schnelltest, falls der VCC-Pin nicht sicher identifizierbar ist** (z. B. unbeschriftetes Breakout): SDA direkt gegen SCL messen. Sind beide Pull-ups vorhanden, fließt intern über das VCC-Netz ein Strom SDA → Pull-up 1 → VCC → Pull-up 2 → SCL — das Multimeter zeigt näherungsweise die **Summe** beider Widerstände (z. B. $\sim 9,4\ \text{k}\Omega$ bei zwei 4,7-k Ω -Pull-ups). Weniger eindeutig als Schritt 2/3: Ist nur **ein** Pull-up vorhanden, existiert kein

geschlossener Kreis und die Messung zeigt fälschlich „kein Pull-up“, obwohl einer da ist — im Zweifel immer Schritt 2 (gegen VCC einzeln) verwenden.

KONDENSATOR AUF DER VCC-LEITUNG

Hat das Modul einen Stützkondensator, lädt sich dieser während der Messung kurz auf — die Anzeige beginnt niedrig und steigt dann auf den tatsächlichen Widerstandswert. 1–2 Sekunden warten, bis sich die Anzeige stabilisiert, bevor der Wert abgelesen wird.

ERGEBNIS ANWENDEN

Pull-up bereits vorhanden → **keinen** zusätzlichen auf der Sensormeter-Steckmodul-Platine bestücken (siehe Abschnitt 6 oben, "Situation A"). Kein Pull-up gefunden → 4,7-k Ω -Widerstände wie in "Situation B" ergänzen, bevor das Modul zum ersten Mal gesteckt wird.

7. Wie viele Geräte gehen?

Theoretisch: ca. 112 nutzbare Geräte bei 7-Bit-Adressen (127 Adressen abzüglich reservierter Bereiche). **Praktisch:** wenige bis einige Dutzend, begrenzt durch gleiche Adressen, Leitungslänge, elektrische Kapazität und Stromverbrauch.

SENSORMETER-SPEZIFISCH — AKTUELLE FIRMWARE-GRENZE

Elektrisch könnten mehrere I²C-Module gleichzeitig am Bus hängen (die Kombimodule zeigen das bereits, siehe Abschnitt 3). **Die aktuelle Firmware erwartet aber bewusst genau ein Modul** — `SensorDetector` bricht den Scan beim ersten Treffer ab. Mehrere gleichzeitig erkannte Module gemeinsam auswerten ist als spätere Erweiterung vorgesehen, aktuell aber noch nicht umgesetzt.

8. Typische Fehler

- SDA und SCL vertauscht
- keine gemeinsame Masse
- falsche Spannung (5 V an einem 3,3-V-ESP32 — **Ausnahme bei uns:** RJ45 Pin 8 führt bewusst 5V für Module wie das Relais, siehe `relais-modul.md` — dort korrekt, aber nie mit SDA/SCL/3V3 verwechseln)
- gleiche Adresse mehrfach vorhanden (siehe Kombimodul-Hinweis oben)
- Kabel zu lang (siehe Abschnitt 5)
- falsche/doppelte Pull-up-Widerstände (siehe Abschnitt 6 + Anleitung oben)

9. Nützliche Werkzeuge

- **I²C-Scanner** → zeigt alle gefundenen Geräteadressen (die Sensormeter-Firmware macht das automatisch selbst — beim Boot und seit 2026-07-15 zusätzlich alle 60 Sekunden, siehe `docs/entscheidungen.md` in `sensormeter/repo` bzw. `sensormeter-poe/repo`)
- **Multimeter** → Spannung prüfen, Pull-ups messen (siehe Anleitung oben)
- **Logic Analyzer** → Signale ansehen (für tiefere Fehlersuche, nicht für den Alltag nötig)

Kurz gesagt

I²C ist ein einfacher 2-Draht-Bus für viele kurze Verbindungen. Jedes Gerät hat eine eigene Adresse. Für Sensormeter/Sensormeter PoE gilt: der Bus läuft effektiv bei ~100 kHz, die RJ45-Steckmodule bringen ihre Pull-ups selbst mit (vorher mit dem Multimeter prüfen!), und die Firmware erwartet aktuell genau ein Zusatzmodul gleichzeitig. Bei langen Kabeln oder mehreren Geräten müssen Busgeschwindigkeit, Pull-ups und Topologie sorgfältig geplant werden.